## Project Report: Generative Search System for Insurance Policy Documents

### 1. Objectives

The primary objective of this project is to develop a robust generative search system that can effectively and accurately answer questions from various insurance policy documents. The system will leverage advanced natural language processing (NLP) techniques and LlamaIndex framework to retrieve and generate relevant answers from a corpus of insurance policy documents.

### 2. Design

#### System Design

The system is designed to ingest, preprocess, and index insurance policy documents, allowing for efficient retrieval and generation of answers to user queries. The overall architecture includes the following components:

1. **Data Collection and Preprocessing**: Gather and preprocess insurance policy documents (HDFC Policies that was provided in the stub notebook).
2. **Indexing**: Use LlamaIndex to index the documents for efficient retrieval.
3. **Query Engine**: Implement a query engine to handle user queries and generate responses.
4. **Response Pipeline**: Create a pipeline to format and present the generated responses.
5. **Testing and Evaluation**: Develop a testing pipeline to evaluate the system's performance.

#### Modules and Components

- **Data Collection**: Collect a diverse set of insurance policy documents.
- **Data Preprocessing**: Clean and structure the documents for text retrieval and generation.
- **Indexing**: Use LlamaIndex to create a vector store index of the documents.
- **Query Engine**: Implement a query engine using the indexed documents.
- **Response Pipeline**: Format the responses and provide references to the source documents.
- **Testing Pipeline**: Evaluate the system's accuracy and performance using a set of predefined questions.

### 3. Implementation

**Code Overview**

The code is structured into various sections, including data loading, preprocessing, model training, and evaluation. The key steps are:

1. **Import Necessary Libraries**: Install and import required libraries such as LlamaIndex, PyMuPDF, and others.
2. **Mount Google Drive and Set API Key**: Access and set up the environment.
3. **Data Loading**: Load and preprocess the insurance policy documents.
4. **Indexing**: Create a vector store index using LlamaIndex.
5. **Query Engine**: Build the query engine and response pipeline.
6. **Testing Pipeline**: Implement a testing pipeline to evaluate the system.

**Key Functions and Classes**

- **split_text**: Splits text into chunks based on token count.
- **process_single_file**: Processes a single file and splits it into chunks.
- **load_documents**: Loads documents from a directory and processes them in parallel.
- **query_response**: Generates a response based on user input by querying the query engine.
- **initialize_conv**: Initializes a conversation with the chatbot.
- **testing_pipeline**: Conducts a testing pipeline for a series of questions.

**Libraries and Frameworks**

- **LlamaIndex**: Used for indexing and querying documents.
- **PyMuPDF**: Used for reading PDF files.
- **OpenAI**: Used for generating responses.
- **Transformers**: Used for tokenization.
- **NLTK**: Used for stop word removal.
- **Pandas**: Used for data manipulation and analysis.
- **Tabulate**: Used for displaying outputs in table format.

**4. Challenges**

**Technical Challenges**

- **Document Preprocessing**: Handling various formats and structures of insurance policy documents.
- **Indexing Efficiency**: Ensuring efficient indexing and retrieval of large documents.
- **Response Generation**: Generating accurate and relevant responses from the indexed documents.

**Non-Technical Challenges**

- **Time Constraints**: Managing time effectively to implement and test the system.
- **Resource Limitations**: Handling computational limitations during indexing and querying.

## 5. Lessons Learnt

**Technical Lessons**

- **Document Handling**: Gained insights into preprocessing and handling various document formats.
- **NLP Techniques**: Improved understanding of advanced NLP techniques for text retrieval and generation.
- **Framework Utilization**: Learned to leverage LlamaIndex framework for efficient document indexing and querying.

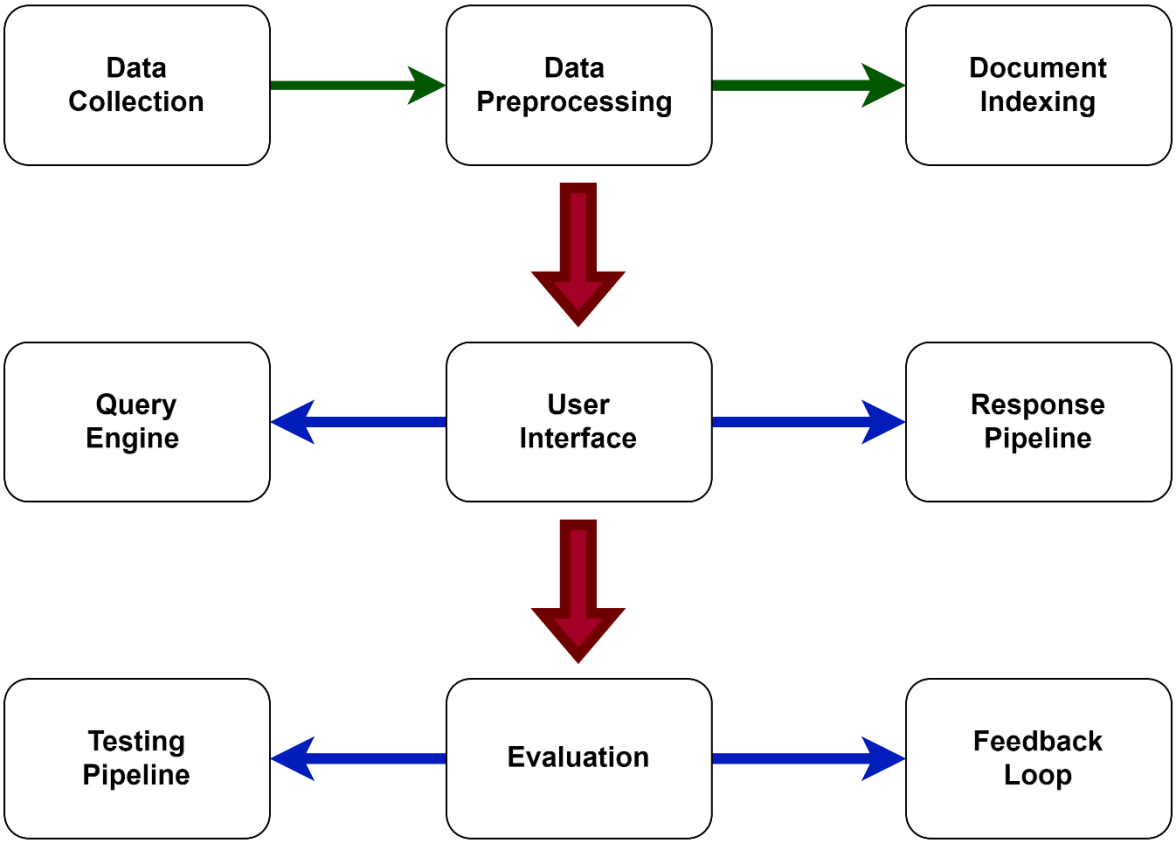
**Non-Technical Lessons**

- **Project Management**: Enhanced project management skills by planning and executing the project within the given timeframe.
- **Collaboration**: Improved collaboration and communication skills while working with team members and stakeholders.

## 6. Overall System Design

**Architecture Diagram**

The overall system architecture can be represented as follows:




**Layered Design**

- **Data Layer**: Responsible for data storage and retrieval.
- **Business Logic Layer**: Handles the core logic of the application, including indexing and querying.
- **Presentation Layer**: Manages the user interface and user interactions.

**Data Collection and Preprocessing**

- **Data Collection**: Gathered a diverse set of insurance policy documents, including health, life, auto, and home insurance policies.
- **Data Preprocessing**: Pre-processed the documents to ensure they are in a suitable format for text retrieval and generation.

**Indexing and Query Engine**

- **Indexing**: Used LlamaIndex to create a vector store index of the documents.
- **Query Engine**: Implemented a query engine using the indexed documents to handle user queries and generate responses.

**Testing and Evaluation**

- **Testing Pipeline**: Developed a testing pipeline to evaluate the system's performance using a set of predefined questions.
- **Evaluation**: Conducted user feedback to assess the accuracy and relevance of the generated responses.

We have implemented the architecture in four distinct scenarios:

1. **Without Custom Prompt Template**
2. **With Custom Prompt Template**
3. **Special Use Cases With Custom Prompt Template**
4. **Implementation Using Retriever Router-Based Query Engine**

For each scenario, we have created a chatbot to demonstrate the limitations and improvements as we progress through the steps.

**Chatbots Implemented:**

1. **Without Custom Prompt Template:**
    - **initialize_conv:**
        - *Description:* This basic chatbot serves as the initial implementation without any custom prompt templates. It provides standard responses based on predefined prompts, lacking the flexibility to handle specific user needs effectively.
2. **With Custom Prompt Template:**
    - **CustomInsurancePolicyHelper:**
        - *Description:* This chatbot uses a custom prompt template to offer more tailored and relevant responses. It improves upon the basic implementation by addressing user queries with greater specificity and contextual understanding.
3. **Special Use Cases With Custom Prompt Template:**
    - **print_claim_filing_instructions:**

        - *Description:* This specialized chatbot provides detailed instructions for filing insurance claims. It leverages a custom prompt template to ensure the information is accurate and user-friendly.
    - **recommend_policies:**
        - *Description:* This chatbot offers personalized insurance policy recommendations based on user input. The custom prompt template allows it to consider various factors and preferences to suggest the most suitable policies.
    - **calculate_premium:**
        - *Description:* This chatbot calculates insurance premiums for users. Using a custom prompt template, it takes into account specific user details and policy parameters to provide precise premium estimates.
    - **compare_policies:**
        - *Description:* This chatbot compares different insurance policies for users. The custom prompt template helps it to highlight the key differences and benefits of each policy, aiding users in making informed decisions.
4. **Implementation Using Retriever Router-Based Query Engine:**
    - **InsurancePolicyAssistant:**
        - *Description:* This advanced chatbot utilizes a retriever router-based query engine to handle complex queries and route them to the appropriate module. It integrates the capabilities of the previous chatbots, offering a comprehensive and efficient solution for all insurance-related inquiries.

## Conclusion

The Generative Search System for Insurance Policy Documents successfully leverages advanced NLP techniques and frameworks to provide accurate and relevant answers to user queries. The project demonstrates the potential of using frameworks like LlamaIndex for efficient document indexing and retrieval, offering valuable insights into the implementation and challenges of building a generative search system.